claude-windows / 9f6357d8-0e35-491a-a830-052e094c7f37

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\9f6357d8-0e35-491a-a830-052e094c7f37\subagents\workflows\wf_635348d6-51c\agent-a24a0a2234b095412.jsonl`
- SHA-256: `8091d462781cb9ce7afd34b2cc4727078a242adea7f981f4517065ee7a78455f`
- Source modified: `2026-06-21T13:56:25+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_635348d6-51c`
- session_id: `9f6357d8-0e35-491a-a830-052e094c7f37`

Transcript

1. user (2026-06-21T13:54:04.250Z)

In the engine repo at C:/Users/avidu/Projects/badminton-highlight-indexer, read the compute-backend code that makes the tool Cloud-Run-friendly: backend/compute/__init__.py, backend/compute/base.py, backend/compute/cloud_run.py. With file:line, explain: the ComputeBackend ABC + registry; what cloud_run.py implements (submit/dispatch, poll/await completion, how it detects a finished job ? bucket-poll?); whether it makes REAL Cloud Run API calls or is scaffolding/mocked; how/if it is wired into /api/process or the job worker or a dispatcher (vs the current in-process ThreadPoolExecutor); the config flag(s) that select the compute backend and their DEFAULTS (confirm Cloud Run is default-OFF / in-process is the default). Grep backend/ for where the registry is consulted. Return structured JSON.

2. assistant (2026-06-21T13:54:07.147Z)

I'll analyze the compute-backend code. Let me start by reading the three specified files and searching for how the registry is consulted.

3. user (2026-06-21T13:54:08.096Z)

```
1       """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
2
3       COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
4       ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
``None``
5       otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte
6       identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
``cpu_mock`` nothing
7       changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
needed).
8       """
9
10      from __future__ import annotations
11
12      import os
13      from typing import Any, Optional
14
15      from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
16      from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
17
18      __all__ = [
19          "ComputeBackend", "ComputeJobSpec", "ComputeResult", "get_compute_backend",
20      ]
21
22
23      def _storage_dict(config: Any) -> dict:
```

```
24          storage = getattr(config, "storage", None)
25          if storage is None:
26              return {}
27          if hasattr(storage, "model_dump"):
28              return storage.model_dump()  # type: ignore[no-any-return]
29          return dict(storage) if isinstance(storage, dict) else {}
30
31
32      def get_compute_backend(
33          config: Any,
34          *,
35          run_client: Optional[Any] = None,
36          policy: ExecutionPolicy = DEFAULT_POLICY,
37          token: Optional[str] = None,
38      ) -> Optional[ComputeBackend]:
39          """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
40          in-process (the default-OFF path).
41
42          Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
43          Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
44          / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
45          pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).
46          """
47          target = getattr(getattr(config, "deployment", None), "compute_target", "") or ""
48          if target != "cloud_run":
49              return None  # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
50
51          # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
52          from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
53
54          job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
55          region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
56          project = os.environ.get("GOOGLE_CLOUD_PROJECT")
57          client = run_client if run_client is not None else GoogleCloudRunJobClient()
58          return CloudRunCompute(
59              run_client=client, job_name=job_name, region=region, project=project,
60              storage_config=_storage_dict(config), policy=policy, token=token,
61          )
62
```

4. user (2026-06-21T13:54:08.597Z)

```
1       """The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.
2
3       COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
4       (``backend/storage/``): a ``ComputeBackend`` decides **where** the pure
DETECT?WINDOW?STITCH
5       core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
6       (cloud-serving) ? without the core ever branching on the profile (D1).
7
8       The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
9       non-local backend. Every other target (``""`` / ``local_gpu`` / ``cpu_mock``) keeps the
worker's
10      existing in-process path **byte-identical** ? the factory returns ``None`` (= "run
inline").
11      """
12
13      from __future__ import annotations
14
```

```
15      from abc import ABC, abstractmethod
16      from dataclasses import dataclass, field
17      from typing import Optional
18
19
20      @dataclass(frozen=True)
21      class ComputeJobSpec:
22          """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
(re)compute
23          from the SAME input bytes, so the output always lands at
``<out_uri>/outputs/<video_id>/`` ?
24          deterministic regardless of where the job ran."""
25
26          video_uri: str                          # input (gs://? for cloud; a path/local://
for local)
27          out_uri: str                            # output ROOT (gs://? bucket for cloud)
28          segmenter: Optional[str] = None
29          # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
frozen/hashable.
30          config_overrides: tuple[str, ...] = ()
31
32
33      @dataclass(frozen=True)
34      class ComputeResult:
35          """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code
36          (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch
timeout or an
37          empty/unreadable completion marker** (the cloud path only). ``video_id`` is known
once the job's
38          done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
landed."""
39
40          returncode: int
41          outputs_uri: str = ""
42          video_id: str = ""
43          report: dict = field(default_factory=dict)
44
45
46      class ComputeBackend(ABC):
47          """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
48          its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
49          execution identically."""
50
51          @abstractmethod
52          def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
53              """Run ``spec`` to completion and return its ``ComputeResult``."""
54
```

5. user (2026-06-21T13:54:09.438Z)

```
1       """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2       for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4       Flow (the cloud-serving compute path):
5         1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6            --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7         2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8            completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
```

```
status, so
9            it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10           cold-start is absorbed by the async poll.
11        3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
12           ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
13
14      The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
15      so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
16
17      The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
18      with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
19      owner-verified on the M7 real run.
20      """
21
22      from __future__ import annotations
23
24      import json
25      import logging
26      import os
27      import tempfile
28      import uuid
29      from abc import ABC, abstractmethod
30      from typing import Any, List, Optional
31
32      from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
33      from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy,
poll_until
34      from backend.storage import fetch, parse_uri
35      from backend.storage import get_storage
36
37      LOG = logging.getLogger("compute.cloud_run")
38
39      # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
40      # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
41      _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
42          "deployment.compute_target=local_gpu",
43          "indexing.detector_impl=native",
44      )
45
46
47      class CloudRunJobClient(ABC):
48          """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
49
50          Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
51          overrides the job's container ``args`` for this execution and starts it."""
52
53          @abstractmethod
54          def run_job(self, job_name: str, argv: List[str], *, region: str,
55                      project: Optional[str] = None) -> str:
56              """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
57              execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
58
59
60      class GoogleCloudRunJobClient(CloudRunJobClient):
```

```
61          """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
62          a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
63
64          def run_job(self, job_name: str, argv: List[str], *, region: str,
65                      project: Optional[str] = None) -> str:
66              try:
67                  from google.cloud import run_v2  # type: ignore[attr-defined]
68              except Exception as exc:  # pragma: no cover - real SDK not present in CI
69                  raise RuntimeError(
70                      "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
71                      "control plane (it is intentionally NOT a CI/test dependency)."
72                  ) from exc
73              client = run_v2.JobsClient()  # pragma: no cover - exercised only on the real M7
run
74              name = client.job_path(project, region, job_name)
75              overrides = run_v2.RunJobRequest.Overrides(
76
container_overrides=[run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)]
77              )
78              op = client.run_job(request=run_v2.RunJobRequest(name=name,
overrides=overrides))
79              return getattr(op, "metadata", None) and getattr(op.metadata, "name", "") or
"execution"
80
81
82      class CloudRunCompute(ComputeBackend):
83          """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
84
85          def __init__(self, *, run_client: CloudRunJobClient, job_name: str, region: str,
86                       project: Optional[str] = None, storage_config: Optional[dict] = None,
87                       policy: ExecutionPolicy = DEFAULT_POLICY,
88                       token: Optional[str] = None,
89                       container_overrides: Optional[tuple[str, ...]] = None) -> None:
90              self._client = run_client
91              self._job_name = job_name
92              self._region = region
93              self._project = project
94              self._storage_config = storage_config or {}
95              self._policy = policy
96              # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
97              # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
98              self._token = token
99              self._container_overrides = (
100                 _DEFAULT_CONTAINER_OVERRIDES if container_overrides is None else
tuple(container_overrides)
101             )
102
103         def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
104             out_root = spec.out_uri.rstrip("/")
105             token = self._token or uuid.uuid4().hex
106             done_uri = f"{out_root}/_dispatch/{token}.done"
107             argv: List[str] = ["infer", "--video-uri", spec.video_uri, "--out-uri",
spec.out_uri,
108                                "--done-uri", done_uri]
109             if spec.segmenter:
110                 argv += ["--segmenter", spec.segmenter]
111             for kv in (*spec.config_overrides, *self._container_overrides):
112                 argv += ["--set", kv]
113
114             execution = self._client.run_job(self._job_name, argv, region=self._region,
```

```
115                                         project=self._project)
116             LOG.info("cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
117                      self._job_name, execution, done_uri)
118
119             marker = poll_until(lambda: self._read_marker(done_uri), self._policy,
120                                 label="cloud-run-infer", logger=LOG)
121             video_id = str(marker.get("video_id", ""))
122             # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
123             # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
124             rc = int(marker.get("returncode", 1))
125             if not video_id:
126                 LOG.warning("cloud-run: job=%s completion marker present but
empty/unreadable (no "
127                             "video_id) ? treating as failure", self._job_name)
128                 rc = rc or 1
129             outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
130             report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
131             LOG.info("cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id,
rc)
132             return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report)
133
134         # --- storage-backed completion signal (bucket-poll) ---
135         def _read_marker(self, done_uri: str) -> Optional[dict]:
136             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
137             ref = parse_uri(done_uri)
138             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
139                 return None
140             return self._read_json(done_uri) or {}  # empty dict still terminates the poll
(present == done)
141
142         def _read_json(self, uri: str) -> dict:
143             # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
144             # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
145             # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
146             try:
147                 ref = parse_uri(uri)
148                 dst = os.path.join(tempfile.mkdtemp(prefix="cr-read-"), ref.name or
"obj.json")
149                 local = fetch(uri, dst, config=self._storage_config)
150                 with open(local, "r", encoding="utf-8") as f:
151                     data: Any = json.load(f)
152                 return data if isinstance(data, dict) else {}
153             except Exception:
154                 LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
155                 return {}
156
```

6. assistant (2026-06-21T13:54:12.136Z)

Now let me find where the registry/factory is consulted and how it wires into the request/job flow. Let me search across the backend.

7. user (2026-06-21T13:54:14.418Z)

```
backend\api\uploads.py:23:from backend.utils.hashing import compute_video_id  # canonical file-
identity hashing
backend\main.py:30:from backend.utils.hashing import compute_video_id  # canonical file-identity
hashing
```

```
backend\compute\base.py:8:The seam is **default-OFF**: only ``deployment.compute_target ==
"cloud_run"`` selects a
backend\compute\cloud_run.py:14:The dispatched container is forced to run **INLINE**
(``compute_target=local_gpu`` + native detector)
backend\compute\cloud_run.py:32:from backend.compute.base import ComputeBackend, ComputeJobSpec,
ComputeResult
backend\compute\cloud_run.py:40:# (compute_target=local_gpu disarms the cmd_infer dispatch
guard; native = the L4 WASB runner.)
backend\compute\cloud_run.py:42:    "deployment.compute_target=local_gpu",
backend\compute\__init__.py:3:COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is
the seam: it returns a
backend\compute\__init__.py:4:``ComputeBackend`` ONLY when a non-local
``deployment.compute_target`` is selected, and ``None``
backend\compute\__init__.py:15:from backend.compute.base import ComputeBackend, ComputeJobSpec,
ComputeResult
backend\compute\__init__.py:19:    "ComputeBackend", "ComputeJobSpec", "ComputeResult",
"get_compute_backend",
backend\compute\__init__.py:32:def get_compute_backend(
backend\compute\__init__.py:39:    """Return the compute backend for
``config.deployment.compute_target``, or ``None`` to run
backend\compute\__init__.py:42:    Only ``compute_target == "cloud_run"`` returns a non-None
backend (a ``CloudRunCompute``). Its
backend\compute\__init__.py:47:    target = getattr(getattr(config, "deployment", None),
"compute_target", "") or ""
backend\compute\__init__.py:52:    from backend.compute.cloud_run import CloudRunCompute,
GoogleCloudRunJobClient
backend\config\models.py:351:    compute_target: Literal["", "local_gpu", "cloud_run",
"cpu_mock"] = ""
backend\config\presets.py:30:# Each profile maps 1:1 to its canonical compute_target. Used to
apply the preset and to warn
backend\config\presets.py:31:# when config.json explicitly overrides compute_target into a state
inconsistent with the profile.
backend\config\presets.py:39:# that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
backend\config\presets.py:45:        "deployment": {"profile": "truly-local", "compute_target":
"local_gpu"},
backend\config\presets.py:52:        "deployment": {"profile": "cloud-serving",
"compute_target": "cloud_run"},
backend\config\presets.py:58:        "deployment": {"profile": "cpu-mock", "compute_target":
"cpu_mock"},
backend\config\loader.py:167:            ct = merged["deployment"].get("compute_target")
backend\config\loader.py:170:                # An active profile with no explicit compute_target
(e.g. config.json blanked it
backend\config\loader.py:172:                # active profile's compute_target (it stays
meaningful for the M2+ ComputeBackend).
backend\config\loader.py:173:                merged["deployment"]["compute_target"] = canonical
backend\config\loader.py:176:                    "[CONFIG] deployment profile %r normally uses
compute_target=%r but "
backend\config\loader.py:180:            logger.info("[CONFIG] deployment profile ACTIVE: %s
(compute_target=%s)",
backend\config\loader.py:181:                        profile,
merged["deployment"].get("compute_target"))
backend\worker\__main__.py:28:from backend.compute import ComputeJobSpec, get_compute_backend
backend\worker\__main__.py:40:from backend.utils.hashing import compute_video_id
backend\worker\__main__.py:119:    container runs this SAME cmd_infer INLINE (compute_target
forced local_gpu) ? never re-dispatch.
backend\worker\__main__.py:162:    # M6: a cloud compute_target DISPATCHES to a Cloud Run Job
instead of running in-process.
backend\worker\__main__.py:163:    # DEFAULT-OFF: get_compute_backend returns None for
""/local_gpu/cpu_mock ? fall through to the
backend\worker\__main__.py:165:    # dispatcher forces compute_target=local_gpu), so it never
recursively re-dispatches.
backend\worker\__main__.py:166:    compute = get_compute_backend(config)
```

8. user (2026-06-21T13:54:15.019Z)

```
backend\api\job_worker.py:21:from concurrent.futures import ThreadPoolExecutor
backend\api\job_worker.py:30:_job_executor: Optional[ThreadPoolExecutor] = None
backend\api\job_worker.py:33:def _get_executor() -> ThreadPoolExecutor:
backend\api\job_worker.py:37:        _job_executor = ThreadPoolExecutor(max_workers=1,
thread_name_prefix="jobworker")
backend\worker\__main__.py:28:from backend.compute import ComputeJobSpec, get_compute_backend
backend\worker\__main__.py:126:    spec = ComputeJobSpec(
backend\worker\__main__.py:131:        result = compute.run_infer(spec)
backend\compute\__init__.py:15:from backend.compute.base import ComputeBackend, ComputeJobSpec,
ComputeResult
backend\compute\__init__.py:19:    "ComputeBackend", "ComputeJobSpec", "ComputeResult",
"get_compute_backend",
backend\compute\base.py:21:class ComputeJobSpec:
backend\compute\base.py:47:    """Backend-agnostic compute. ``run_infer`` dispatches ONE
inference job and returns only once
backend\compute\base.py:52:    def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
backend\compute\cloud_run.py:32:from backend.compute.base import ComputeBackend, ComputeJobSpec,
ComputeResult
backend\compute\cloud_run.py:97:        # None ? a fresh uuid4 per run_infer call (prod);
injected ? deterministic (tests).
backend\compute\cloud_run.py:103:    def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
backend\eval\gemini_refine.py:121:    with
concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as ex:
backend\eval\gemini_refine.py:186:    with
concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as ex:
backend\pipeline\segmenters\gemini.py:252:        with
concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as executor:
backend\pipeline\segmenters\trajectory_hybrid.py:344:            with
concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as executor:
backend\pipeline\segmenters\yolo_hybrid.py:128:            extract_executor =
concurrent.futures.ThreadPoolExecutor(max_workers=prefetch_workers)
backend\pipeline\segmenters\yolo_hybrid.py:307:        with
concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as executor:
```

9. assistant (2026-06-21T13:54:17.669Z)

The key wiring is in `backend/worker/__main__.py`. Let me read the relevant section and the `job_worker.py` to understand the in-process path and how the two connect.

10. user (2026-06-21T13:54:19.044Z)

```
100         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
101         (returns True, ZERO work) when no deployment.profile is selected.
102
103         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
104         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
105         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
106         no-op of work, not just of output."""
107         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
108         cuda = probe_cuda_available() if profile else None  # torch-free on the default-OFF
path
109         try:
110             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
111             return True
112         except StartupCheckError:
113             return False  # already logged at ERROR by enforce_startup_checks
114
115
116     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
117         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
```

```
        outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
        container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.

        A remote job that never writes a completion marker (a failed/hung container) makes
the bucket
        poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
(remote dispatch
        timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
        from backend.infrastructure.execution_policy import PolicyTimeout

        spec = ComputeJobSpec(
            video_uri=args.video_uri, out_uri=args.out_uri,
            segmenter=args.segmenter, config_overrides=tuple(args.set or ()),
        )
        try:
            result = compute.run_infer(spec)
        except PolicyTimeout as e:
            logger.warning("[worker] remote compute did not complete (no marker within
budget): %s", e)
            return 4
        logger.info("[worker] remote compute done: video_id=%s rc=%d outputs=%s",
                    result.video_id, result.returncode, result.outputs_uri)
        return result.returncode


    def _write_done_marker(done_uri: str, video_id: str, returncode: int, segment_count:
int,
                           status: str, storage_cfg: dict, scratch: str) -> None:
        """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
        dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
        try:
            marker = {"video_id": video_id, "returncode": returncode,
                      "segment_count": segment_count, "status": status}
            local = os.path.join(scratch, "_done.json")
            with open(local, "w", encoding="utf-8") as f:
                json.dump(marker, f)
            storage.put(local, done_uri, config=storage_cfg)
            logger.info("[worker] wrote completion marker -> %s", done_uri)
        except Exception as e:  # never fail a good run because the marker push hiccuped
            logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)


    def cmd_infer(args: argparse.Namespace) -> int:
        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        if not _run_startup_checks(config):
            return 2

        # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
        # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
        # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
        # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
        compute = get_compute_backend(config)
        if compute is not None:
            return _dispatch_remote(compute, args)

```

```
170         storage_cfg = config.storage.model_dump()
171
172         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
173         os.makedirs(scratch, exist_ok=True)
174         config.output.output_dir = scratch
175
176         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
177         local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
178         print(f"[worker] pulled {args.video_uri} -> {local_video}")
179
180         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
181         video_id = compute_video_id(local_video)
182
183         # 3. VALIDATE (same registry as the main CLI).
184         val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
185         failures = [r for r in val_results if not r.passed]
186         db = Database(os.path.join(scratch, config.output.db_name))
187         db.add_video(video_id, local_video, "failed" if failures else "processed",
188                      [r.model_dump() for r in val_results])
189
190         segments: List[dict] = []
191         segmenter_actual = None
192         segmentation_ran = False
193         status = "failed"
194         try:
195             if failures:
196                 print("[worker] validation FAILED: "
197                       + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
198                 return 2
199             seg_name = args.segmenter or config.indexing.default_segmenter
```

11. user (2026-06-21T13:54:19.619Z)

```
1       """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3).
2
3       Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
4       The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
5       that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
6       re-claimed when due ? with no central scheduler (the DB is the clock).
7
8       The monkeypatch contract these functions rely on (and that tests depend on):
9       every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
10      `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
11      `backend.main` module object at call-time (`import backend.main as _main`), never as a
12      bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
13      ...)` still intercepts the inter-function calls (e.g. a parked job arms the *patched*
14      `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
lazily
15      inside the functions also avoids the circular import at module load (main.py imports
this
16      module; this module must not import main at import time).
17      """
18      import logging
19      import threading
20      import traceback
21      from concurrent.futures import ThreadPoolExecutor
22      from typing import Any, Dict, Optional
23
24      logger = logging.getLogger(__name__)
25
```

```
    # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
    # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
    # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
    # already parallelize their AI handoff internally via their own pools.
    _job_executor: Optional[ThreadPoolExecutor] = None


    def _get_executor() -> ThreadPoolExecutor:
        """Lazy module-global executor; tests monkeypatch this for inline execution."""
        global _job_executor
        if _job_executor is None:
            _job_executor = ThreadPoolExecutor(max_workers=1,
thread_name_prefix="jobworker")
        return _job_executor


    class JobWaiting(Exception):
        """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
        WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`",
        NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
        releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
        is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.

        The wiring is additive: the production segmenter path never raises this today (zero
        behaviour change), so a job runs exactly as before. The hook is what a later slice
        (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
        """

        def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
            super().__init__(f"job parked for {delay_sec:.0f}s")
            self.delay_sec = max(0.0, float(delay_sec))
            self.progress = progress


    def _job_to_request(job: Dict[str, Any]):
        """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
        re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
        the original submit. The row stores exactly the ProcessRequest fields."""
        import backend.main as _main
        return _main.ProcessRequest(
            video_path=job["video_path"],
            player_pool=job.get("player_pool"),
            max_frames=job.get("max_frames"),
            segmenter=job.get("segmenter"),
        )


    def _run_claimed_job(job: Dict[str, Any]) -> None:
        """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
        terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).

        Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
        the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
        result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
        job self-scheduled and will be re-claimed when `next_poll_at` is due.
        """
        import backend.main as _main
        db_instance = _main.db_instance
        job_id = job["id"]
```

```
84          req = _main._job_to_request(job)
85          try:
86              result = _main._execute_processing(
87                  req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
88              if result.get("video_id"):
89                  db_instance.set_job_video_id(job_id, result["video_id"])
90              db_instance.mark_job_done(job_id, result)
91              _maybe_record_job_cost(job, result)  # M8 cost ledger (default-OFF)
92          except _main.JobWaiting as w:
93              # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
94              logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
95              db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
96          except Exception as e:
97              logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
98              db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
99
100
101     def _maybe_record_job_cost(job: Dict[str, Any], result: Dict[str, Any]) -> None:
102         """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
103         ``config.billing.enabled``. When on, compute the job's cost from ``result['usage']``
104         (= {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the versioned
pricebook, and
105         persist it.
106
107         ? **Honest gap (M8 scope):** the pipeline does **not** emit ``result['usage']`` yet,
so even with
108         ``billing.enabled=True`` AND a real calibrated pricebook, every recorded cost is
**0** until a
109         FOLLOW-UP wires the worker to emit usage (gpu/wall/egress/gemini). M8 lands the
ledger + pricebook
110         + the recording seam; the usage-emission step is separate (the metering hooks on the
real run).
111         With the placeholder pricebook the cost is 0 regardless.
112
113         Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job``
is the
114         claimed row (carries ``owner_id``); ``result`` is the pipeline output."""
115         import backend.main as _main
116
117         billing_cfg = getattr(_main.global_config, "billing", None)
118         if not billing_cfg or not getattr(billing_cfg, "enabled", False):
119             return
120         try:
121             usage = (result or {}).get("usage") or {}
122             _main.db_instance.record_job_cost(
123                 job["id"], usage=usage, pricebook_version=billing_cfg.pricebook_version,
124                 gpu_type=usage.get("gpu_type"), compute_backend=(result or
{}).get("compute_backend"),
125                 owner_id=job.get("owner_id"), margin=billing_cfg.margin,
126             )
127             logger.info("Recorded cost for job %s (pricebook=%s).", job["id"],
billing_cfg.pricebook_version)
128         except Exception as e:  # never wedge a completed job on bookkeeping
129             logger.warning("Cost recording for job %s failed (non-fatal): %s",
job.get("id"), e)
130
131
132     def _drain_due_jobs() -> int:
133         """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
134         `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
135
136         Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
137         up by a FUTURE drain once due; we schedule that follow-up drain via
```

```
`_schedule_drain` so
138         a single-worker box keeps making progress with no central timer (the DB is the
clock).
139         Returns the number of jobs that reached a terminal/parked state this tick.
140         """
141         import backend.main as _main
142         ran = 0
143         while True:
144             job = _main.db_instance.claim_due_job()
145             if job is None:
146                 break
147             ran += 1
148             _main._run_claimed_job(job)
149         # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
150         # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
151         # on the next submit/drain. Best-effort ? never raises into the worker.
152         _main._schedule_next_drain_if_waiting()
153         return ran
154
155
156     def _schedule_next_drain_if_waiting() -> None:
157         """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
158         self-scheduled job resumes with no further client submit. The drain itself re-checks
159         due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
160         import backend.main as _main
161         try:
162             delay = _main.db_instance.seconds_until_next_due()
163         except Exception as e:  # pragma: no cover - defensive; never wedge the worker
164             logger.error(f"Could not compute next-due delay: {e}")
165             return
166         if delay is None:
167             return  # nothing waiting
168         # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
169         _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)))
170
171
172     def _arm_wake_timer(delay_sec: float) -> None:
173         """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
174         scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
175         monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
176         import backend.main as _main
177         timer = threading.Timer(delay_sec, lambda:
_main._get_executor().submit(_main._drain_due_jobs))
178         timer.daemon = True
179         timer.start()
180
181
182     #: A parked job further out than this is re-checked by a capped wake rather than one
long
183     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
184     _MAX_DRAIN_WAKE_SEC = 60.0
185
```

12. assistant (2026-06-21T13:54:24.075Z)

Now I understand the critical wiring gap. The `get_compute_backend` seam is consulted ONLY in `backend/worker/__main__.py::cmd_infer` (the standalone CLI worker), NOT in the API's `job_worker.py` / `_execute_processing` (the in-process ThreadPoolExecutor path used by

`/api/process`). Let me confirm by checking `_execute_processing` in `backend/main.py` and the `/api/process` route, and verify the config defaults.

13. user (2026-06-21T13:54:26.083Z)

```
148:def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
151:    This is the former synchronous /api/process body, unchanged in behavior, now run on
268:@app.post("/api/process", status_code=202)
269:def process_video(req: ProcessRequest):
394:@app.post("/api/query")
405:@app.post("/api/compile")
457:        # video ingested via /api/process from a URI, or a plain local path. Resolve it to a
LOCAL file
459:        # for a bucket URI (the same seam the process path uses, _execute_processing
main.py:159).
```

14. user (2026-06-21T13:54:26.916Z)

```
335         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
336         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
337
338         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
339         behaves exactly as before this section existed. A profile is opt-in (``config.json``
340         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
341         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
342         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
343         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
344
345         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
346
347         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
348         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
349         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
350         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
351         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
352
353
354     class BillingConfig(BaseModel):
355         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
356
357         **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
NULL =
358         today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
the versioned
359         ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
360         version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
owner
361         inserts a real, calibrated version and points ``pricebook_version`` at it.
362
363         ? Even with ``enabled=True`` + a real pricebook, recorded cost stays **0** until a
follow-up
364         wires the worker to emit per-job ``usage`` (gpu/wall/egress/gemini) ? M8 lands the
ledger +
365         pricebook + the recording seam; usage emission (the metering hooks) is a separate
step."""
366
367         enabled: bool = False
```

```
368         pricebook_version: str = "placeholder-v0"   # the seeded $0 version; owner flips to
a real one
369         fx_inr_per_usd: float = 83.0                  # USD?INR display rate (Bangalore
market)
```

15. user (2026-06-21T13:54:27.566Z)

```
1       """Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING
M1).
2
3       A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
4       config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
5       knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
6
7       Precedence (lowest ? highest), realised by the loader::
8
9           built-in model defaults  <  profile preset  <  config.json  <  env  <  worker --set
10
11      This module is **pure data + pure functions** ? no IO, and (critically) **no torch
import at
12      module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
13      (``config.json`` ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); with no
14      profile selected the loader is byte-identical to the pre-M1 behaviour (``profile=""`` ==
15      today). Auto-detect only ever **suggests** a profile ? it never auto-applies one (D15:
cloud
16      spend is never entered silently).
17      """
18
19      from __future__ import annotations
20
21      import copy
22      import os
23      from typing import Any, Mapping, Optional
24
25      # Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal``
mirrors
26      # these and ``tests/test_config_deployment.py`` pins parity so the two can never drift.
27      PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28      COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
29
30      # Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to
warn
31      # when config.json explicitly overrides compute_target into a state inconsistent with
the profile.
32      COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
33          "truly-local": "local_gpu",
34          "cloud-serving": "cloud_run",
35          "cpu-mock": "cpu_mock",
36      }
37
38      # A preset is a partial, nested override dict over the model defaults. It touches ONLY
knobs
39      # that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
40      # ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones
extend
41      # these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
42      # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
43      PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44          "truly-local": {
45              "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46              # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
box).
47              # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
48              "indexing": {"skip_ai_handoff": True},
```

```
49              "storage": {"backend": "local"},
50          },
51          "cloud-serving": {
52              "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
53              # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
(D11).
54              "indexing": {"detector_impl": "native", "skip_ai_handoff": False},
55              "storage": {"backend": "gcs"},
56          },
57          "cpu-mock": {
58              "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
59              # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
profile.
60              "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
61              "storage": {"backend": "local"},
62          },
63      }
64
65
66      def is_known_profile(profile: str) -> bool:
67          """True for a non-empty, recognised profile name."""
68          return bool(profile) and profile in PROFILE_PRESETS
69
70
```

16. assistant (2026-06-21T13:54:30.682Z)

Now let me confirm the `/api/process` ? `_execute_processing` path to be certain the compute backend is NOT consulted there, and check the execution_policy poll primitive briefly.

17. user (2026-06-21T13:54:32.136Z)

```
148     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
149         """The full hash ? validate ? segment ? report pipeline for one video.
150
151         This is the former synchronous /api/process body, unchanged in behavior, now run on
152         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
153         the sync endpoint used to return (UI compatibility); the per-video observability
154         report is still emitted in `finally:` and grafted as response["reports"].
155
156         `progress` is an optional callable(stage: str) used to surface coarse job progress.
157         Raises on unexpected internal errors ? the caller records those as a job failure
158         (after this function has already restored the pre-run segments, see below).
159         """
160         notify = progress or (lambda stage: None)
161
162         notify("hashing")
163         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
164         # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
165         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
166         # consumers (hash / validators / segmenter) use the resolved local path.
167         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
168         video_id = compute_video_id(local_video)
169         was_reingest = db_instance.get_video(video_id) is not None
170
171         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
172         notify("validating")
173         logger.info(f"Running validations for {req.video_path}...")
174         val_results, val_context = registry.run_all_with_context(local_video, global_config)
175         failures = [r for r in val_results if not r.passed]
```

```
176
177         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
178         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
179         seg_name = req.segmenter or default_seg
180         segmenter_actual = None
181         segmentation_ran = False
182         response: Optional[Dict[str, Any]] = None
183         try:
184             if failures:
185                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
186                 # status; it no longer destroys the video's prior good segments.
187                 db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
188                 response = {
189                     "video_id": video_id,
190                     "status": "failed",
191                     "message": "Video failed validation checks.",
192                     "results": [r.model_dump() for r in val_results],
193                 }
194                 return response
195
196             # 2. Run segmenter & indexer
197             logger.info("[API] Video passed checks. Segmenting and indexing...")
198             db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
199
200             segmenter_cls = segmenter_registry.get(seg_name)
201             if not segmenter_cls:
202                 # Submit-time validation already 400s unknown names; this is the defensive
203                 # in-worker path (e.g. config default pointing at an unregistered
segmenter).
204                 available = list(segmenter_registry.list_available().keys())
205                 response = {
206                     "video_id": video_id,
207                     "status": "failed",
```

18. user (2026-06-21T13:54:32.658Z)

```
268     @app.post("/api/process", status_code=202)
269     def process_video(req: ProcessRequest):
270         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
271
272         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
273         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
274         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
275         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
276         """
277         allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
278         try:
279             validate_input_path(req.video_path, allowed_root=allowed_root)
280             # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
281             # scheme, which we map to a clean 400 (an unknown scheme must never 500).
282             input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
283         except ValueError as e:
284             raise HTTPException(status_code=400, detail=str(e)) from e
285         # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
286         # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
287         # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
```

```
288         # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
289         # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
290         # rejects a non-local URI as out-of-root.
291         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
292             raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
293         if req.segmenter and not segmenter_registry.get(req.segmenter):
294             available = list(segmenter_registry.list_available().keys())
295             raise HTTPException(
296                 status_code=400,
297                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
298             )
299
300         job_id = uuid.uuid4().hex
301         if not db_instance.create_job(job_id, req.video_path, req.segmenter,
302                                       req.player_pool, req.max_frames):
303             raise HTTPException(status_code=500, detail="Failed to persist job record.")
304         _get_executor().submit(_drain_due_jobs)
305         return JSONResponse(
306             status_code=202,
307             content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
308         )
309
310
311     @app.get("/api/jobs/{job_id}/cost")
312     def get_job_cost(job_id: str):
313         """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of
truth (matches
314         GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.
``cost_usd`` is
315         null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
316         cost = db_instance.get_job_cost(job_id)
317         if cost is None:
```

19. assistant (2026-06-21T13:54:36.603Z)

I now have a complete picture. The `/api/process` ? `_drain_due_jobs` ? `_run_claimed_job` ? `_execute_processing` path never consults `get_compute_backend`; it always runs in-process via the `ThreadPoolExecutor`. The compute seam is wired ONLY into the standalone CLI worker (`backend/worker/__main__.py::cmd_infer`). Let me verify the `poll_until` primitive that powers completion detection.

20. user (2026-06-21T13:54:38.529Z)

```
8:  ``max_wait_sec`` deadline).
39:class PolicyTimeout(TimeoutError):
44:class ExecutionPolicy:
48:    op_timeout_sec: float = 120.0         # HARD per-attempt deadline ? the hang-killer that
was missing
70:DEFAULT_POLICY = ExecutionPolicy()
78:def run_bounded(fn: Callable[[], T], policy: ExecutionPolicy = DEFAULT_POLICY, *,
125:def poll_until(check: Callable[[], Optional[T]], policy: ExecutionPolicy = DEFAULT_POLICY,
*,
```

21. user (2026-06-21T13:54:41.592Z)

```
44      class ExecutionPolicy:
45          """Declarative resilience knobs, attachable per job/op (JSON-serialisable; see
EXECUTION_POLICY.md)."""
46          mode: WaitMode = WaitMode.WAIT_AND_RESUME
47          poll_every_n_sec: float = 10.0
```

```
48          op_timeout_sec: float = 120.0        # HARD per-attempt deadline ? the hang-killer
that was missing
49          max_wait_sec: float = 1800.0         # total wall budget for a poll loop
50          max_retries: int = 5                 # retries (=> max_retries+1 attempts) for
failures/hangs
51          backoff_base_sec: float = 3.0        # exponential: backoff_base * 2**attempt
52          on_timeout: str = "reschedule"       # reschedule | partial | fail  (consumed by
P2/P3)
53          resumable: bool = True
54
55          def to_dict(self) -> dict:
56              d = asdict(self)
57              d["mode"] = self.mode.value
58              return d
59
60          @classmethod
61          def from_dict(cls, d: dict) -> "ExecutionPolicy":
62              known = {f for f in cls.__dataclass_fields__}  # type: ignore[attr-defined]
63              kw = {k: v for k, v in d.items() if k in known}
64              if "mode" in kw and not isinstance(kw["mode"], WaitMode):
65                  kw["mode"] = WaitMode(kw["mode"])
66              return cls(**kw)
67
68
69      #: The sane default (also what a job gets if it declares no policy).
70      DEFAULT_POLICY = ExecutionPolicy()
71
72
73      def _backoff_sec(policy: ExecutionPolicy, attempt: int) -> float:
```

22. user (2026-06-21T13:54:42.402Z)

```
125     def poll_until(check: Callable[[], Optional[T]], policy: ExecutionPolicy =
DEFAULT_POLICY, *,
126                    label: str = "op", logger: Optional[logging.Logger] = None,
127                    sleep: Callable[[float], None] = _time.sleep,
128                    monotonic: Callable[[], float] = _time.monotonic) -> T:
129         """Poll ``check()`` every ``poll_every_n_sec`` until it returns a non-``None``
value, or raise
130         ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is
still in progress.
131
132         Polls log at DEBUG (no INFO spam); the wait start + resolution + give-up log at
INFO/WARNING.
133         """
134         log = logger or LOG
135         start = monotonic()
136         polls = 0
137         log.info("%s: waiting (poll_every=%.0fs, max_wait=%.0fs)",
138                  label, policy.poll_every_n_sec, policy.max_wait_sec)
139         while True:
140             result = check()
141             if result is not None:
142                 log.info("%s: ready after %d poll(s) / %.0fs", label, polls, monotonic() -
start)
143                 return result
144             elapsed = monotonic() - start
145             if elapsed >= policy.max_wait_sec:
146                 log.warning("%s: still not ready after %.0fs (max_wait) ? giving up", label,
elapsed)
147                 raise PolicyTimeout(f"{label} not ready within
max_wait_sec={policy.max_wait_sec}s")
148             polls += 1
149             log.debug("%s: poll %d, %.0fs elapsed, not ready", label, polls, elapsed)
150             sleep(policy.poll_every_n_sec)
```

23. assistant (2026-06-21T13:54:44.530Z)

I now have a complete and verified picture. Let me compile the structured findings.

24. user (2026-06-21T13:55:37.400Z)

Output does not match required schema: root: must NOT have additional properties

25. assistant (2026-06-21T13:55:40.363Z)

The schema rejected my extra `cloudRunBackendNote` key. Let me fold that content into `mockedVsReal`/`cloudRunBackend` and resubmit with only the allowed properties.

26. user (2026-06-21T13:56:18.953Z)

Structured output provided successfully

27. assistant (2026-06-21T13:56:25.076Z)

Structured output submitted successfully. Summary of what I verified across the three compute files plus the wiring:

- The seam is an ABC + lazy factory, not a Registry singleton: `backend/compute/base.py` (ComputeBackend ABC, ComputeJobSpec/ComputeResult dataclasses) and `backend/compute/__init__.py::get_compute_backend` (returns None for everything except `compute_target == "cloud_run"`).
- `backend/compute/cloud_run.py` dispatches via an injected `CloudRunJobClient`, then bucket-polls a storage done-marker (not the Cloud Run API status) via `poll_until`; the real GCP client exists but is `pragma: no cover` and never exercised in CI.
- Critical finding: the seam is wired only into the standalone CLI worker (`backend/worker/__main__.py::cmd_infer`), NOT into `/api/process`, which always runs in-process on the `max_workers=1` ThreadPoolExecutor in `backend/api/job_worker.py`.
- Confirmed default-OFF: `deployment.profile` and `deployment.compute_target` both default to `""` (`backend/config/models.py:348,351`), so in-process is the default.